Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026-27	**Semester:**	I
Name of the Student:	V. Mokshmika	**Roll Number:**	A2112625S2187
Name of the Lab:	Full stack	**Timings:**	9:40-12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva- Voce (2Marks)			
01/07/26	01	4	3	5	2	14		
08/07/26	02	4	3	5	2	14		
15/07/26	03	4	4	5	2	15		
22/07/26	04	4	4	5	2	15		
29/07/26	05	4	3	5	2	14		
05/8/26	06	4	4	5	2	15		
12/8	07	4	3	5	2	14		
19/8	08	4	3	5	2	14		
2/9	09	4	4	5	2	15		
9/9	10	4	4	5	2	15		
23/9	11	4	4	5	2	15		
30/9	12	4	3	5	2	14		

Name of the Faculty: Prasanna Priya

Faculty Signature:

Introduction to HTML and CSS :

HTML (Hyper Text Markup Language) is the standard markup language used to create and the structure content on webpages. it defines that basic structure of a webpage using elements and tags such as headings, paragraphs, links, images lists, tables, forms and other multimedia content HTML documents are interepted by web Browsers to display the content in a structured and an meaningful manner HTML provides the structure and semantic organization of a webpage.

CSS (cascading style sheets) is a style sheet language used to control the presentation and visual apperance of HTML documents it is used to define properties such as colours, fronts, background, spacing, bolders, allignment, dimension and layouts CSS also supports responsive design, allowing webpages to adapt to different screen sizes and devices, CSS can be applied using inline internal, or external stylesheets.

HTML and CSS work together to develop webpg, where html provides the structure and content, while CSS provides the styling layout visual presentation the combination of both forms are

are foundation of ... user-friendly web interfaces.

Difference between MERN stack and full stack

MERN stack is an javascript based technology stack used for developing modern web applications

MERN stands for MongoDB, Express.js, React.js Node.js, Mango DB - used for database managment Express.js - handles the backend framework and API's React.js - develop frontend user interface and node.js provide the server side runtime environ-ment

Full stack development refers to development of both the frontend backend of web application along with D.B integration

A full stack worker or an developer works with user interface and implementing the server side logic managing databases

HTML Components

HTML Tags — Define the structure and element of a webpage

Headings — used to display titles & heading using <h1> to <h6>

paragraphs — used to display text using <p> tag

Links — used to navgate btw webpages of using <a> tag

images - used to display images using <img> tag

Lists - used to create oldered & unolderd lists using <ol>

Tables - used to olganize data in rows and columns

forms - used to collect users i/p using elements as text box button checkbox

div & span - used to group and olganize webpage content

semantic element are <header> <nav> <selects <article> <footer>

Provide meaningful webpage structure

CSS components

Selectors – select HTML elements to supply styles

Properties – Define visual characteristics of elements

Values – specify setting of css property

Colors – Define text-, bg-, border colours

Front – Control front family, size, style and weight

Background – define the set bg colrs, img

Borders – define border style, width radius

Margin – Controls space outside element

Padding – " space inside element

Layout – Control the arragment of elements using Flexbox, Grid and positioning

HTML vs HTML5

HTML	HTML5
older version of html	Latest major version of html
Limited Multimedia support	Support audio & Vedio
uses external plugins for multimedia	No external plugs need
has fewer semantic tags	Provides mor semantic tags
No <canvas> Element	Support <canvas>
Limited web storage	support Local storage
Complex doctype declaration	simple <!Doctype> HTML

CSS vs CSS3

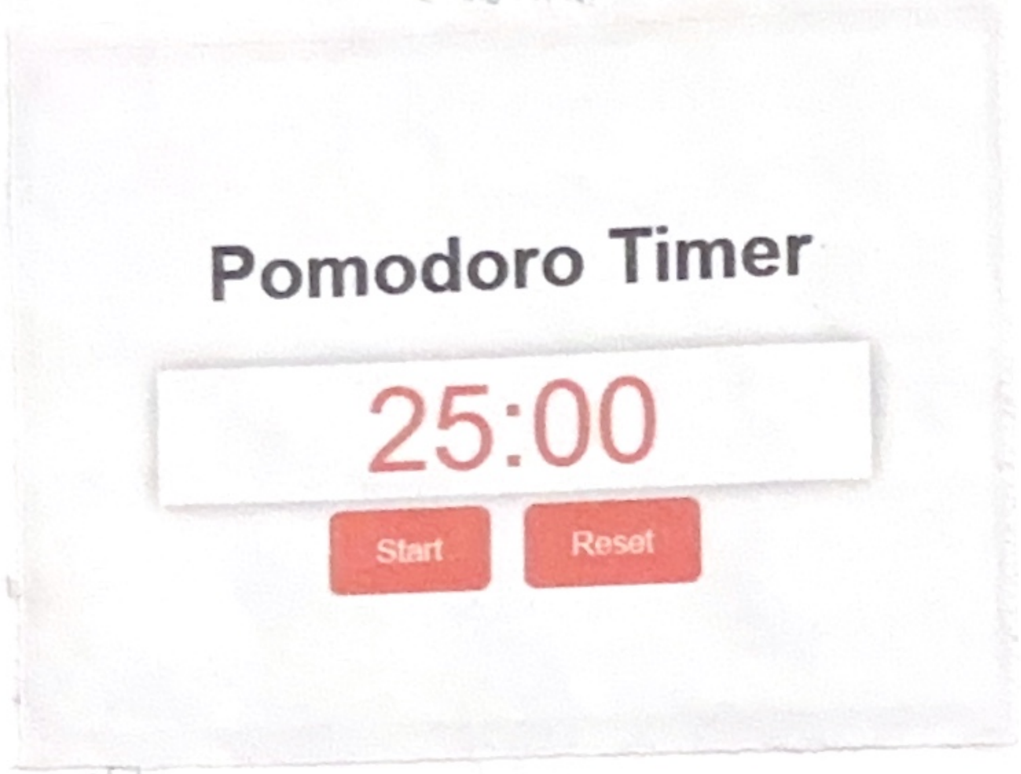
CSS	CSS 3
earlier version of CSS	Adv version CSS
Basic styling features	Adv styling features
Limited animation	support animation and transitions
No flexible layout system	supports flexbox & grid
limited visual effects	support shadows, gradients, effects

Slot Design an static webpage using html & css
Pomodoro timer, using html css

```
<!doctype html>
<html>
<head>
    <title> Pomodoro timer </title>
    <style>
        body{
            font-family : Arial;
            background : #f5f5f5
            text-align : center;
            padding : 50px;
        }
timer {
        width : 300px;
        margin : auto;
        padding : 30 px;
        background: white;
        border radius : 15 px;
        box-shadow: 0 0 10px gray;
}
hi{
    Color : #074 c3c;
}
timer
    front-size: 50px;
    margin : 20px;
}
button {
    padding : 10px 20px;
    margin : 5px;
    border : none
border -radius : 5px
background : #c74c9c;
colour : white;
}
</style>
</head>
<body>
        <div class "timer">
<h1> Pomadoro Timer </h1>
<div class = "time"> 25:00
        </div>
<button> start </button>
<button> reset </button>

    </div>
</body>
</html>
```

o/p :



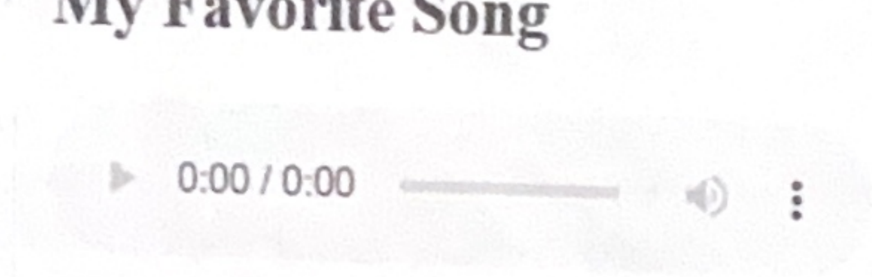
(1) Design an HTML code using audio tags

```
<!Doctype html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
        <h2> My favorite Song </h2>
        <audio Control>
            <source src="song.mp3" type=
"audio/mpeg">
            Your Browseser does not support
              the audio tag
        </audio>
</body>
</html>
```

steps:-

Create a new html file
add basic html structure
add heading using <h2> tag
insert <audio> tag with control attribute
specify the audio file using <source> tag
save, open Html file in web Browser
play audio using display controls.

My Favorite Song



(iii) Design a html code using video tags.

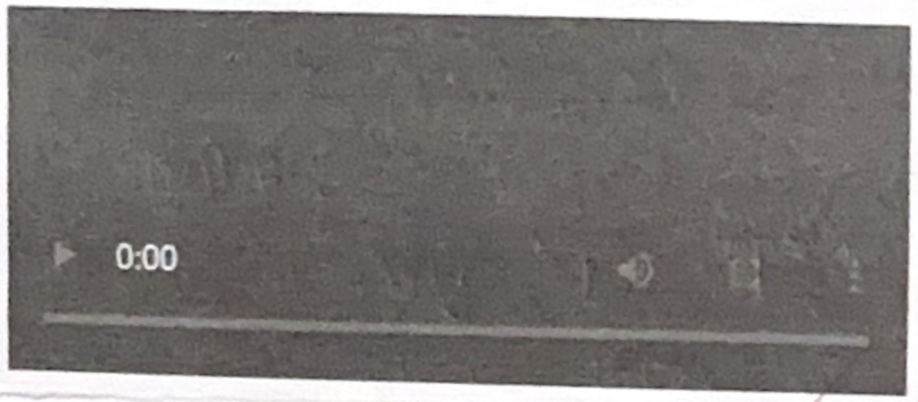
Code:

```
<!Doctype html>
<html>
 <head>
    <title> Video example </title>
</head>
<body>
      <h2> My video </h2>
      <video width = "400" controls>
       <source src = "video.mp4" type = "video/mp4">
      </video>
</body>
</html>
```

steps:

create a new html file
add a basic html structure
add a heading using <h2> tag
insert the <video> tag with control attribute
specify the video file using <source> tag
save the file and open it the web browser
play the video using displayed controls

My Video



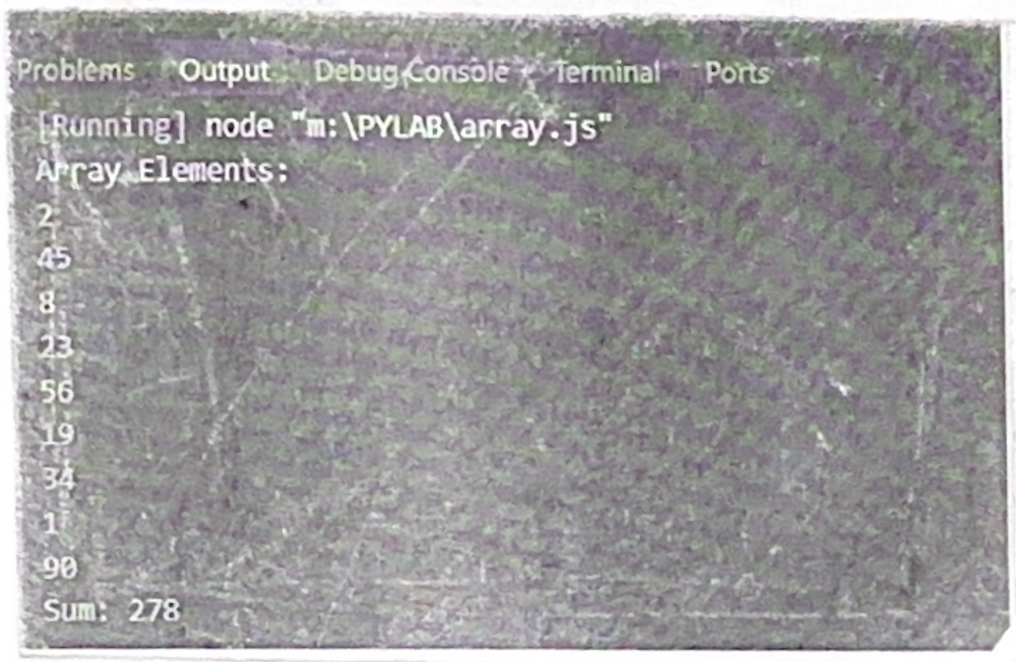
write a program using Javascript
(arrays, functions)

Code

```
let number = [12,45,8,23,56,19,34,67,
                                90,11];

function display Array (arr) {
    console.log ("array Elements : ");
    for (let i=0; i< arr.length ;i++){
        console.log (arr[i]);
    }
}

function findSum (arr) {
    let sum=0;
    for (let i=0; i<arr.length; i++){
        sum += arr[i];
    }
    return sum;
}
```



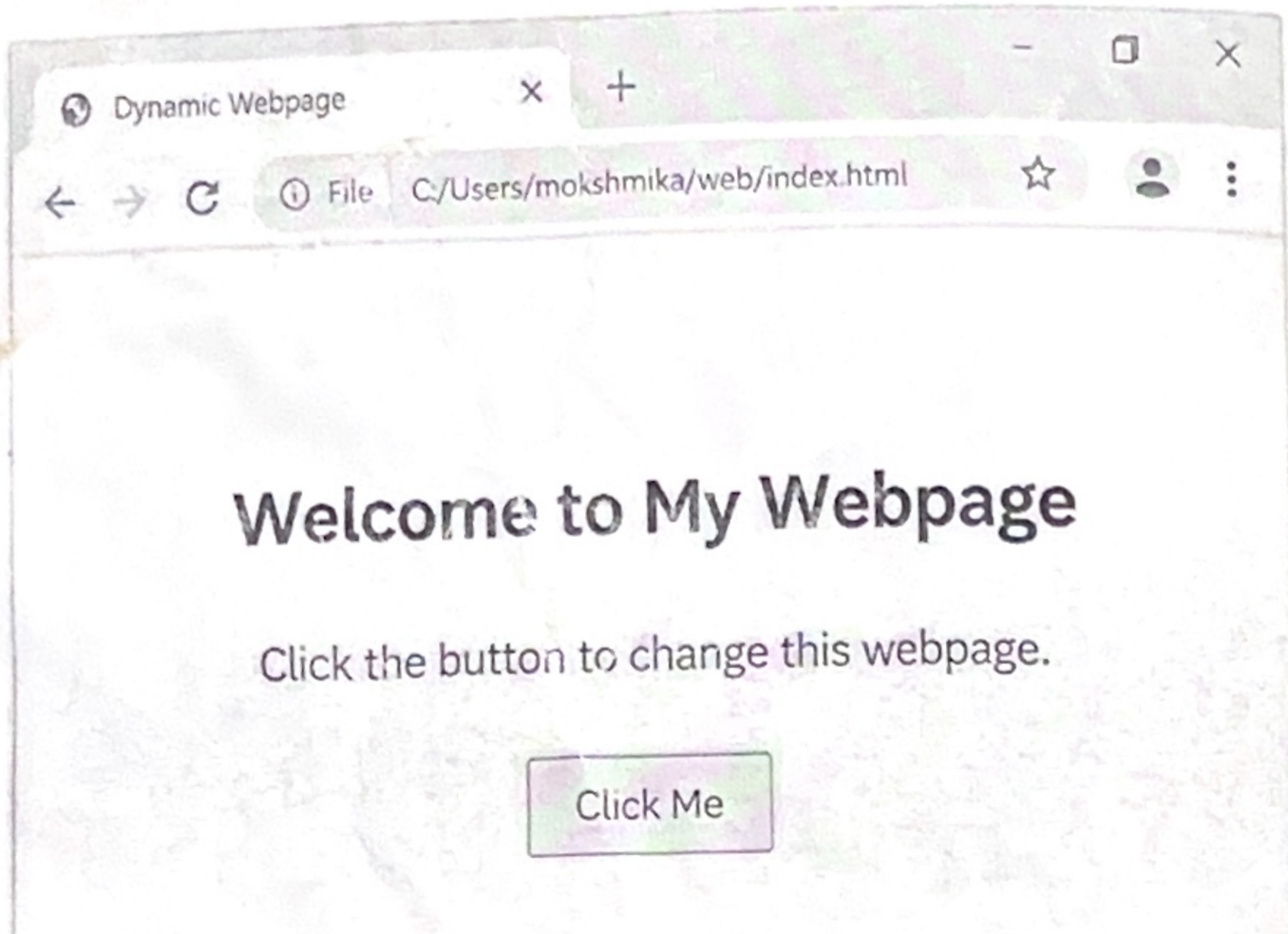
Experiment - 04

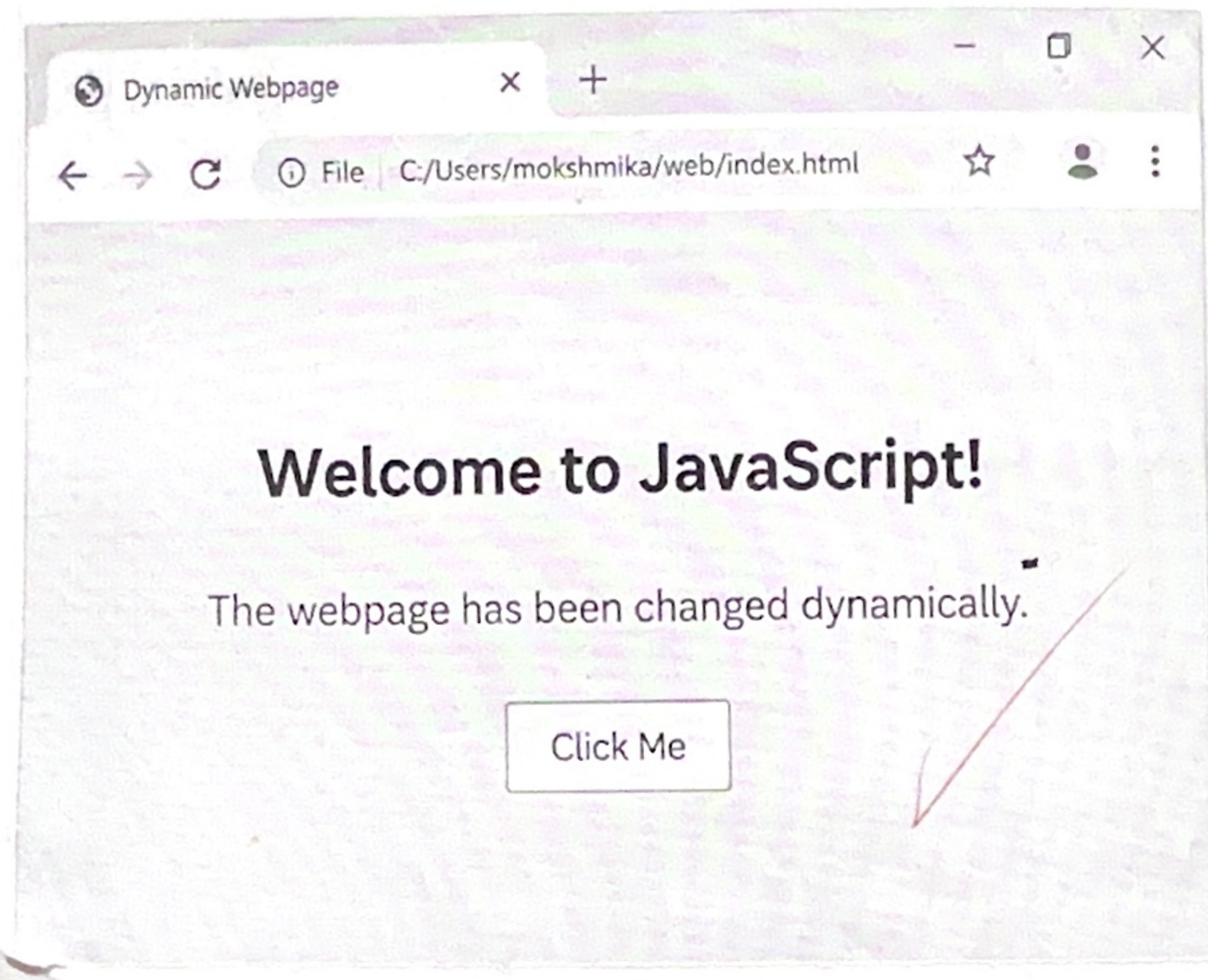
4 Develop dynamic Webpage using Java Script

Program

```
<!Doctype html>
<html>
<head>
<title> Dynamic Webpage </title>
<style>
body{
    text align : center;
    front family : Arial :
    margin top : 100px;
}
button {
    Padding 10px 20px;
    front size : 16px
    cursor : pointer;
}
</style>
</head>
<body>
<h1 id="heading">welcome
    to My webpage </h1>
<p id = "message"> click the button to
        change this webpage : </p>
<button onclick = "change. Page ()">
        click Me </button>

<script>
function change page (){
document. get Element by Id ("heading")
inner html = "Welcome to Java
        sript!";
document. get element By Id ('message')
inner html = "the webpage has
        been changed dynami-
        -cally.";
document body style . background
        colour
        = " Light blue"; }
</script>
</body>
</html>
```





Experiment - 05

05 Implement Modulus in node Js

In node js: you can implement modulus. using two distint systems: Common Js (CJS), which is traditional default. and ES Modulus (ESM) the modern java script standard

1. The common Js Method (default)

Common Js uses module exports to share code and require() to import. the system of back modules synchronusily

a. Create the module (math.js)

```
Const add = (a,b) ⟶ a+b;
Const subtract = (a,b) ⟶ a-b;
Module.exports = {
    add,
    subtract
};
```

b. use the module (math.js)

```
Const math = require('./math');
Console.log (math.add (10,5));
Console.log (math.subtract (10,5));
```

output: 15
3

2. the fs module method (modern standard)

Es module use Export to share code import to use it To make node.js recognize Es module you must either use the mjs file of Extension or add "type" to your package.json

a. Create the module (Calculator.js)

Export const multiply = (a,b) → axb

Export const divide = (a,b) → a/b

Const calculatorName = "supercale"

Export default calculatorName;

b. use the module (main.js)

import calculatorName, {multipy, divide} From "./calculator.js";

Console.log (calculatorName);

Console.log (multiply (5,2);

Console.log (divide (4,2));

output: supercalate

8

2

3. using Buit in Core modules:

Node.js also comes equipped with dozens of built in core modules like for fs file system or path for the faulting that don't need an installation.

we will load them By module names:

```
//comments style
Const fs = require('fs')
// fs module style
import fs form 'fs';
```

Here, are some of the most commonly used ones;

1. fs = file system operations
2. http = HTTP server and client
3. path = file path utilites
4. os - operating system
5. events = event handling
6. util - utility functions
7. stream - stream handling
8. crupto - cryptographic func
9. url - url parsing
10. querystring - url query string handle

Example: Simple HTTP

```
http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end('Hello World!');
}).listen(8080);
```

output: Hello World !

Experiment -06

Develop mini applications using express and node Js

sample mini applications using node Js,

express Js

Student Managment System:

1. add a student
2. view all students
3. delete a student
4. uses express js routes
5. simple html interface

Project structure:

```
student-app/
├── server.js
└── public/
    └── index.html
```

server.js:

```
Const express = require("express");
Const app = express();
const port = 3000;
let students = [];
app.use(express.json());
app.use(express.static("public"));
app.get("/students", (req, res) => {
  res.json(students);
});
//add
app.post("/students", (req, res) => {
  Const student = {
    id: student.length + 1;
    name: req.body.home,
    course: req.body.course
  };
  student.push(student);
  res.json({message: "student added successfully", student: student
  });
});
//delete
app.delete("/student/:id", (req, res) => {
  const id = parseInt(req.params.id);
  student = student.filter(student => student.id !== id);
  res.json({
    message: "student deleted succesfully"});
```

```
console.log("server running at http://
        local host: ${port}");

});
Public index.html;
<!doctype html>
<html>
<head>
<title> student
        managment </title>
</head>
<body>
<h1> student Managment
        system </h1>
<input type="text" id="
        course"
placeholder="Enter course">
<button onclick="add student
()"> add student </button>
<h2> student list </h2>
<ul id: "student list" ></ul>
<script>
function addstudent() {
Let source = document. get element
        by Id ("course");
        value;
fetch ("/students", {
method: "pose",
headers: {
"Content-Type" = " application/
        json"};
body: Json stringfy({
        name: name;
        course: course
        })
then (response → response.json())
then (data => {
        alert (data.message)
document. get Element By id ("name") value
        =" ";
document. get element By id ("course").
        Value = " ";
display students());
});
}
function display()
fetch ("students"). then
        (respons → response json())
then (student) → {
        let list = document.
                get element
        by id ("student list");
```

```
document.
li.innerHtml = student.id + " - " +
                student.name + " - " +
                  student
                course + " <buttons
onclick = 'deletestudent'
          (" + student.id + ")>
                  delete
</button>
list.appendChild(li)
            });
        });
    }
function deleteStudent(id){
    fetch("student/" + id,
        {method "delete"}
then(data => {
        alert(data.message());
        displayStudent();
    });
displayStudents();
</script>
</body>
</html>
```

output:

student Managment system

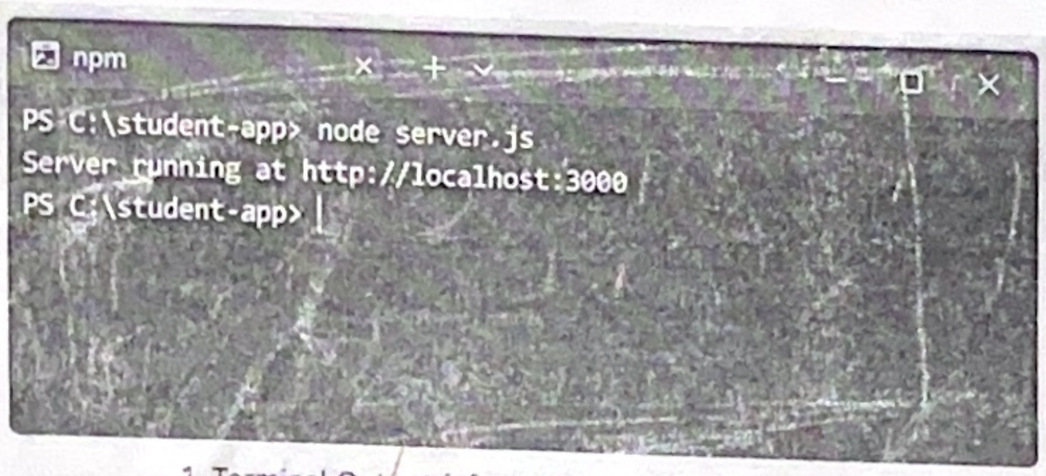
Enter student name

Enter course

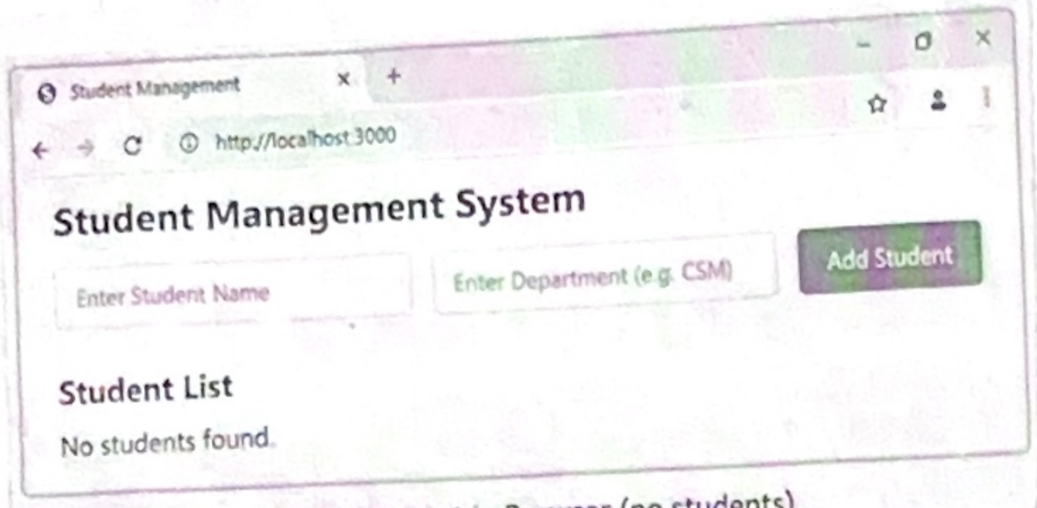
add student

student list

1. Tejeswari - CSM [delete]
2. sita - CSE [delete]

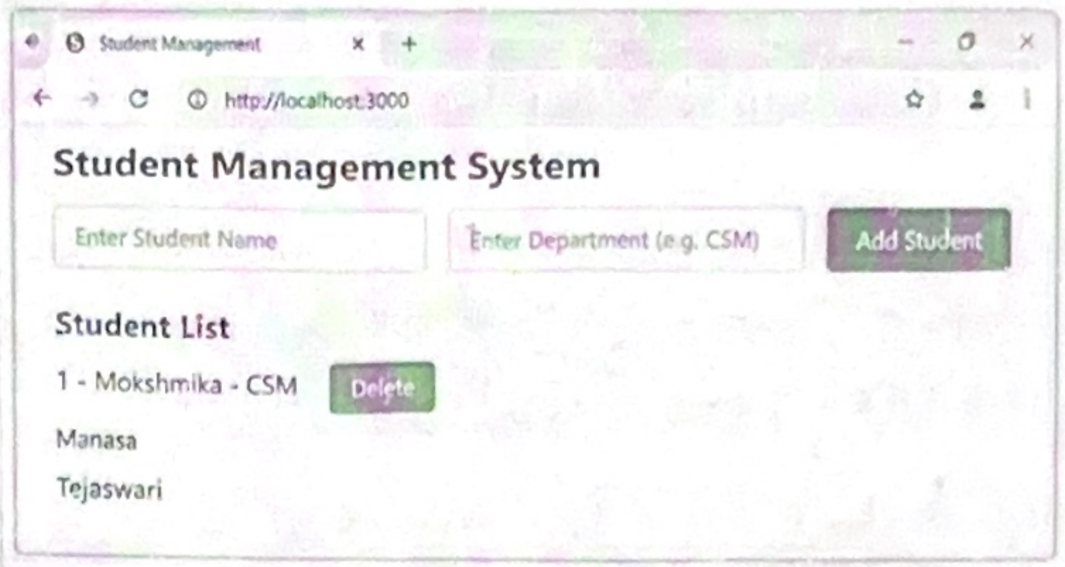


1. Terminal Output (after running `node server.js`)

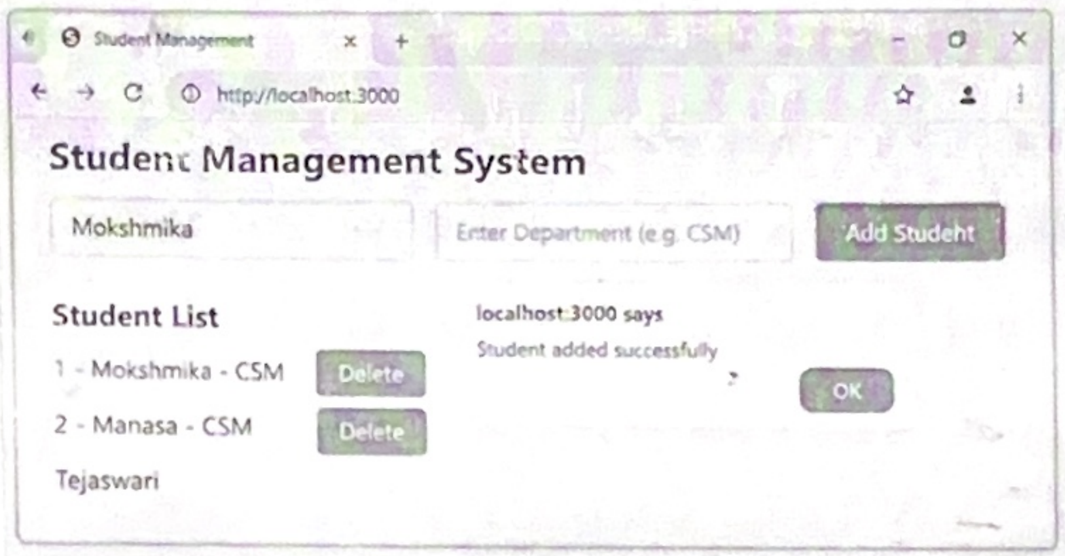


2. Initial Output in Browser (no students)

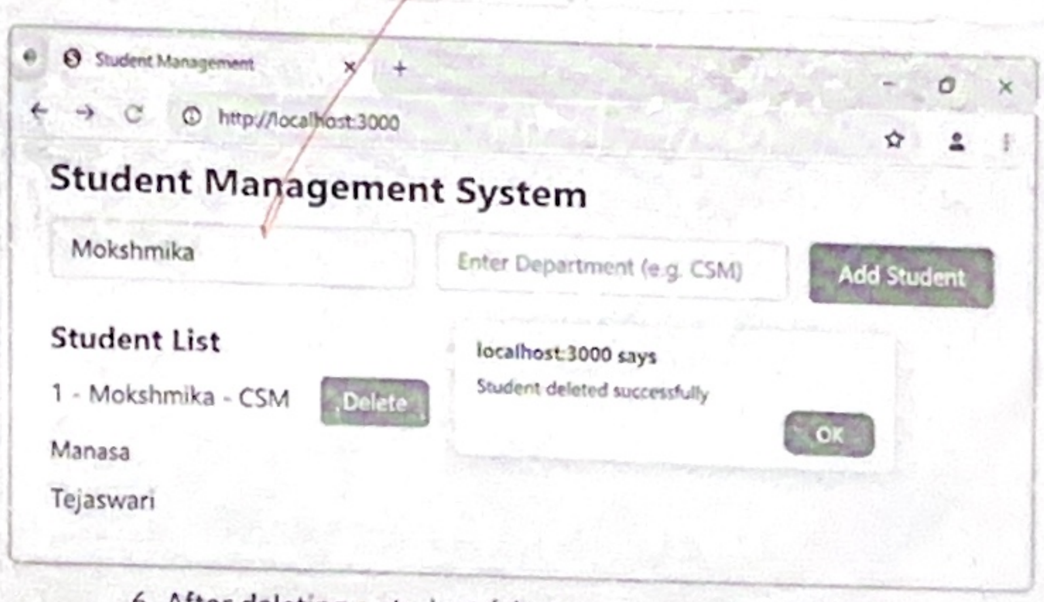
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

experiment - 07

1. create a functional component and class component using react

Aim :- To develop a react application using the functional Components and class components to display student, details, manage state, handle button clicks dynamically update data in web page.

* functional component :-

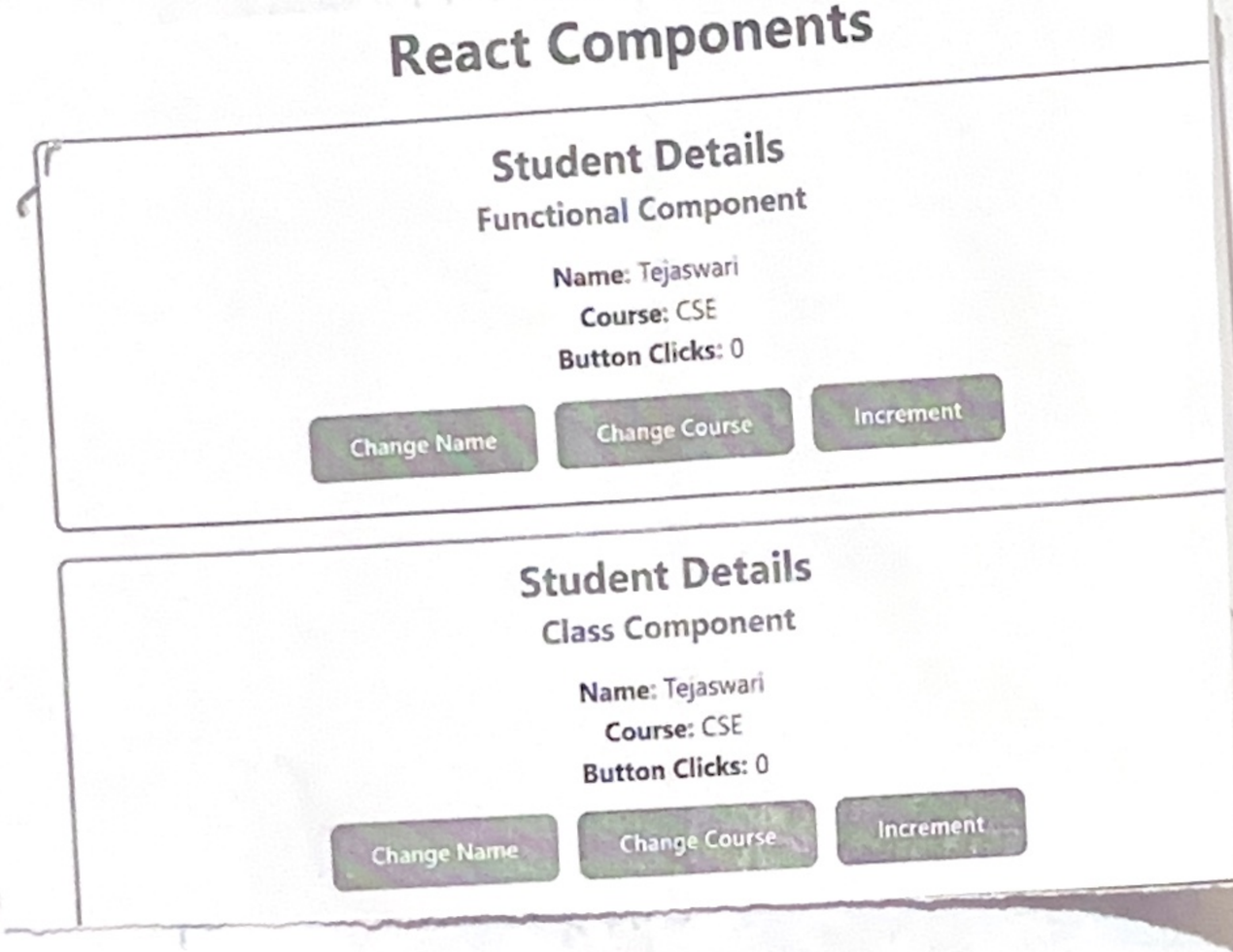
A functional component is an JS function that returns JS to display the user interface uses react hooks as usestate to manage the state and manage update the webpage dynamically

Program :-

```
import react, {use state} form
'react':
function studentFunctional (){
        Const [name, setName] = use state
                                ["Mokshu"]
        Const [Course, setCourse ]= usestate
                                ("CSM");
        Const [count, setCount]. use
                                state (0):
        Const changeName = ()→ {
                setName ("Mokshu")
        };
        Const changeCourse = () → {
                set Course ("CSD")
        };
        Const incrementCount = ()→ {
                Set Count (count+1) : };
        return (
                <div style = {{
                        border : "2px solid blue",
                        Padding : "20px",
                        margin : "20px",
                        border Radius : "10px",
                        text Align : "center"
                }} >
                <h1> Student Details </h1>
                <h2> Functional Component </h2>
                <p><b> Name : </b>{name} </p>
                <p> <b) Course : </b>{course} </p>
                <button · · on click = {change Name}
                        change Name </button>
                <button · on click = {change Course}
                        change Course </button>
```

```
<button onclick = {incrementCount}> Increment
        </button>
    </div>
  );
};
```

Output: when the user click the button, the name, course and count change dynamically without loading the webpage



class Component

A class component is an css Javascript class that extends react Component it uses the render method to return Jsx manages the state state and this.setState()

Program :-

```
import react, { component } from
'react'
class student-class extends Component {
    Constructor (prop) {
        Super (props);
        this.state : {
            name = "Mokshu"
            Course = "Computer science"
            Count 0
        }
    }
change Name = () → {
    this.setState {{
        name :"mokshu"
    }};
};
```

Experiment - 08

implement http methods using JSON Server

Aim: to implement HTTP methods such as GET POST, PUT, MATCH and delete using JSON server to perform CRUD [create, read, update, delete] of operation on student records

Theory:- JSON Server is a tool used to create a fake rest API using a Json file. It allows us to perform HTTP operations without development a backend server form scratch

HTTP methods are used to communicate btw a client and a server

GET - retrives data from the server

POST - adds new data to the server

PUT - replaces as existing records

delete - removes a record for server

a. Create a Json server

step 1: Initiliaze the project

open the VS Code terminal & execute, the following commands

```
mkdir Json server
cd Json server
npm init -y
npm install json server
```

step 2: Create db.json

Create a file named db.json and add the following student data.

```
{
"students": {
  {
   'id' : "1",
   "name" : "Mokshmika"
   "course" : "CSM"
   "marks" :90"
  }
  {
   "id" : "2"
   "name" : "manasa"
   "course" : "CSM"
   "marks" : 90
  }
  {"id" : "3"
   "name" : "Monika"
   "course" : "IT"
   "marks" : 88
  }
```

step 3: Start the JSON server

for the current JSON server version, execute:

npx json.server db.json

The server start at:

http://localhost:3000

the student records can be accessed

http://localhost:3000/students

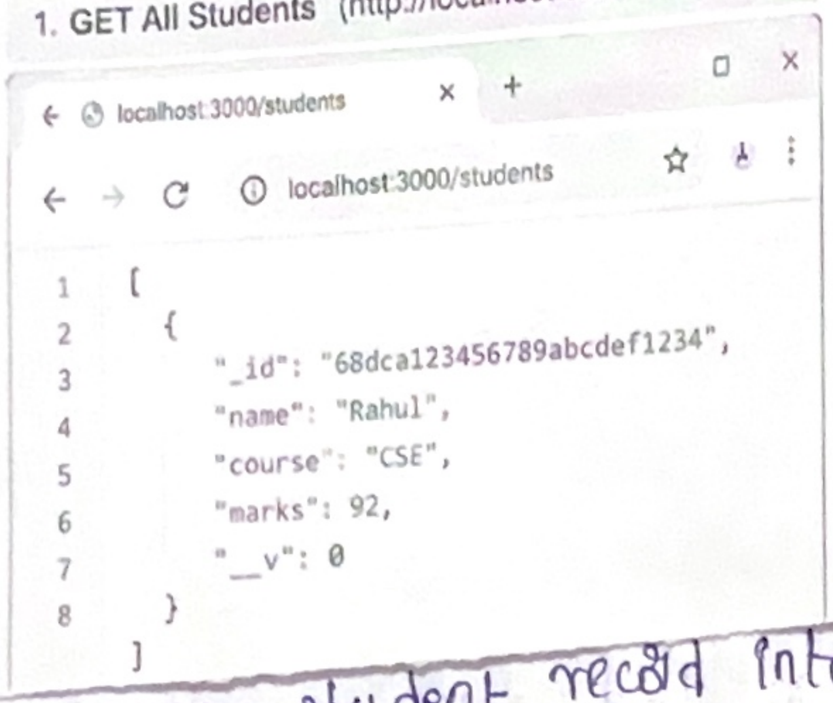
2. implementation of HTTP methods

a. GET method: The get method is used to of retrive student records from JSON server

Code:

```
fetch("http://localhost:3000/students")
.then(response → response.json())
.then(data ⇒ {
        Console.log(data);
})
.catch(error ⇒ {
        Console.log(error);
})
```

1. GET All Students (http://localhost:3000/students)



B. ...s used to insert a new student record into server

Code: Fetch (http://local host :3000/students', {

method : 'POST'

headers : {

'content-Type' : 'application/json

},

body : Json.stringify({

name : 'Rahul',

Course : 'CSE'

marks : 92

})

})

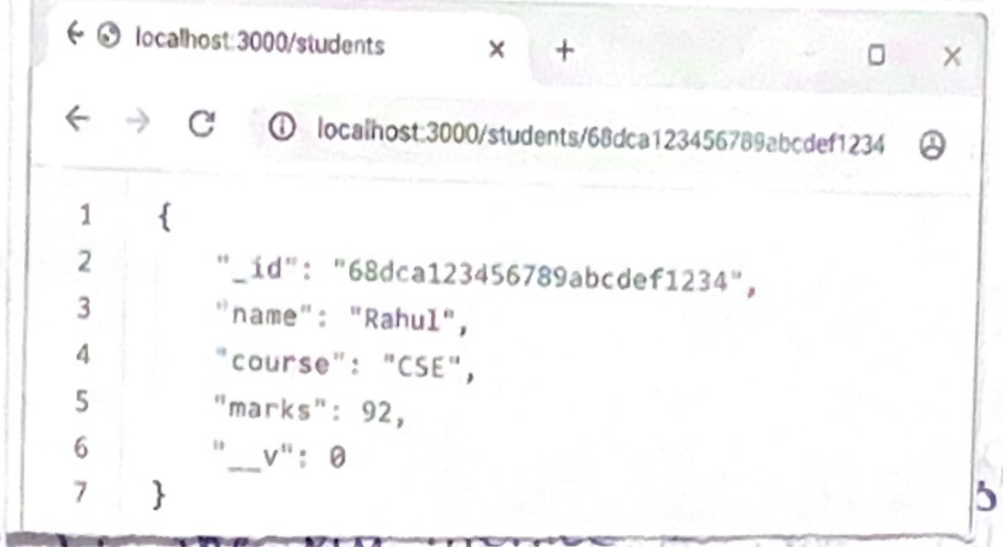
.then (data => {

console.log(data)

})

output :

2. GET Student by ID (http://localhost:3000/students/68dca123456789abcdef1234)



C. put method: ...s replace an exisiting student record updated data.

Code: fetch (http://local host 3000/students/1', {

method: 'PUT'

headers: {

'content-Type' : 'application/json'

},

body Json.stringify({

name : "mokshmitra"

Course : "AIML"

marks : 95

})

})

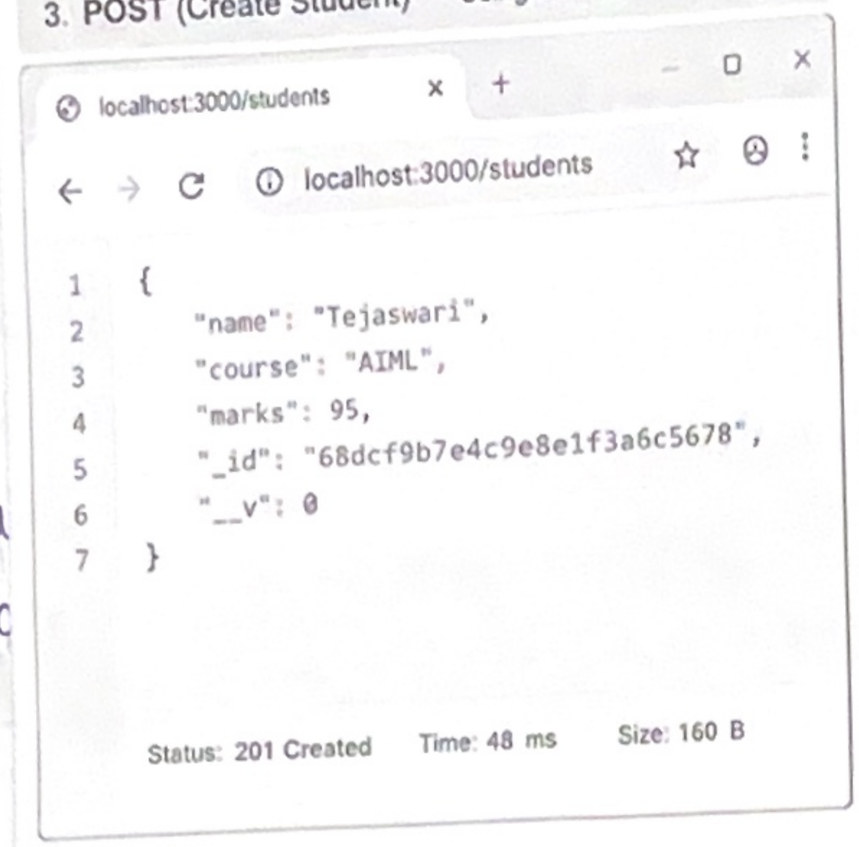
.then (response → response.json())

})

.catch (error => {

Console.log(error) });

3. POST (Create Student) – Using Postman



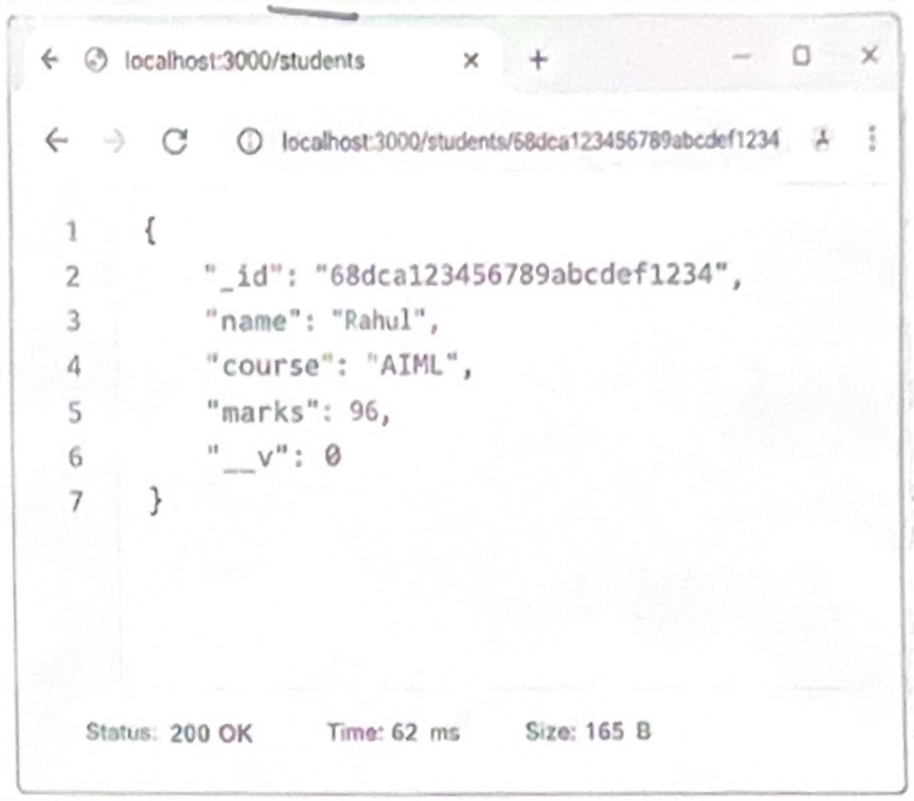
used to
lent record
, (students);

D. Pa
upda
Code:

headers : {
'Content-Type' : 'application/json',
}
body : JSON . stringfy ({
marks : 96 }) })
. then (data ⟹ {
. catch (error ⟹ {
Console . log (error);
});

output :-

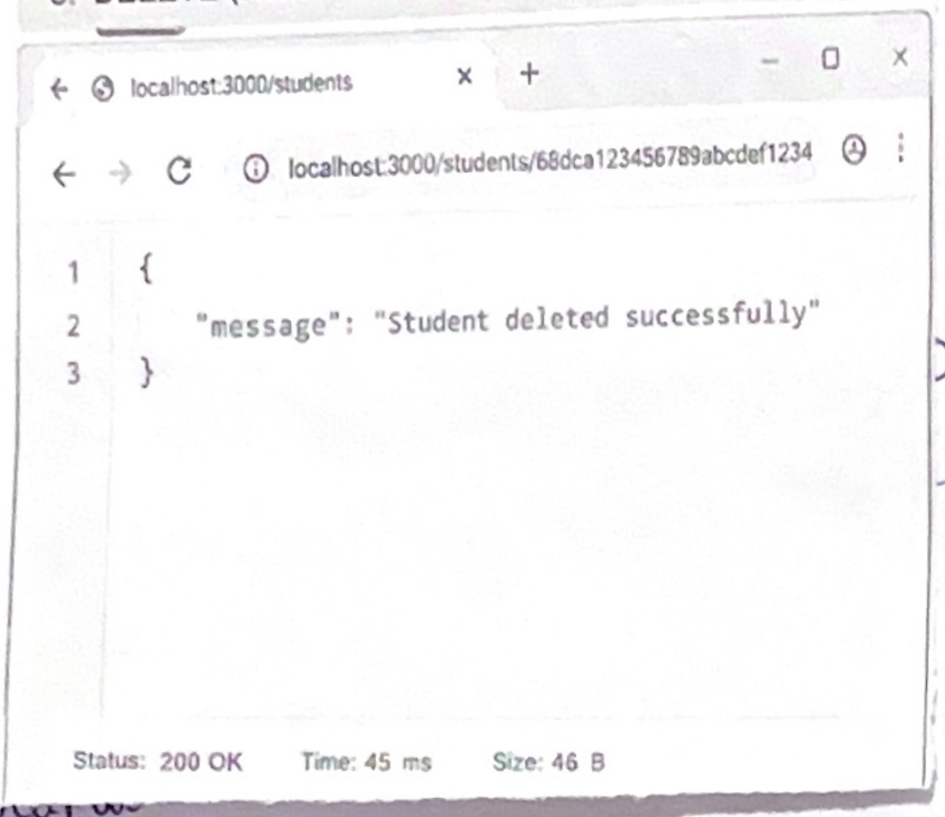
4. PUT (Update Student) – Using Postman



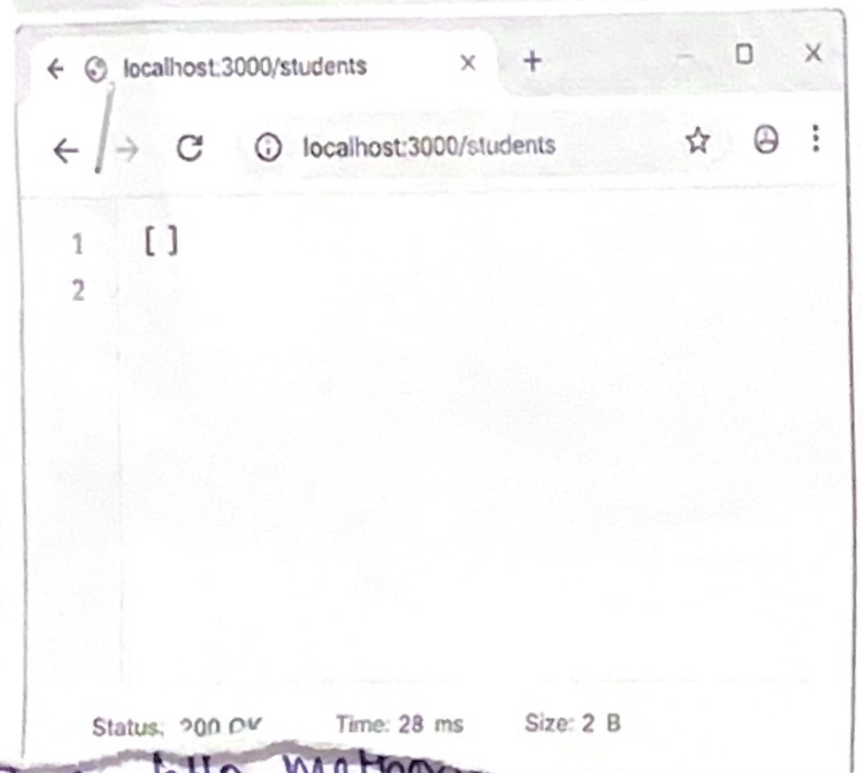
E. delet
to remove an existing
used
server

Code: Fetch (http:// local host · 3000 /students/1s}
method : 'Delete' · })
· catch (error ⟹ {
console . log (error) });
})
then (response ⟹ {
if (response . ok) {
console . log ('student
delete succesfully)
}
})

5. DELETE (Remove Student)



6. GET All Students (After Deletion)



Thus http methods ... t, were been succesfully implemented using Json Server Crud operation were performed on student records the data was retrived, inserted, deleted using http req

Experiment - 09

09. Develop CRUD operations using MongoDB

Aim :- To develop a web application using node.js express.js and mongoDB. (Create, read, update, delete) on student records

Theory :- Mongoose is an object data modeling for mongoDB and node.js It helps define schema interact with collections and performance.

1. installation and project Setup :-

```
mkdir mongoDB.CRUD
cd MongoDB.CRUD
npm.init.y
npm.install express mongoose
```

2. Program ; server.js

```
const mongoose = require ('mongoose');
const app = express();
const Port = 3000;
app.use (express.json())
mongoose connect ('mongoDB' : //127.0.0.1
                  : 27017/.college)
then () → consde.log ("mongoDB connected")
catch (err → consde.log (Error));
```

```
//create student
const . student Schema = new
mangoose schema ({
    name: {
      type: String
      required : true
};
    course : {
      type: string,
      required : true
    };
    marks : {
        type: number
        required : true
```

```
// Create : Add
app.post ("/students ; async (req . res)→ {
    try{
      const student : new Student (req.
                                  body);
    Const . saved student. await student
                                  save();
    res. status (201): /json (saved student)
                  error . message 3);
});
```

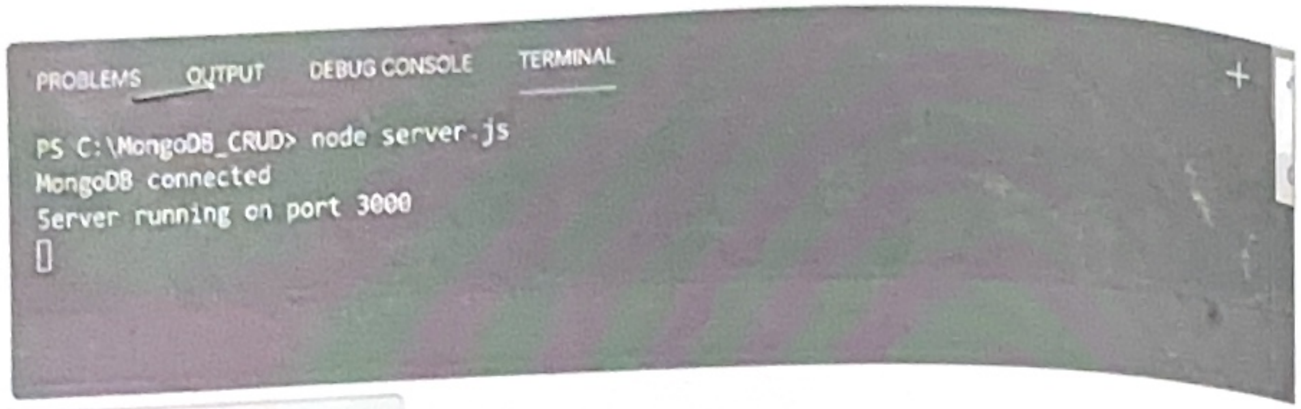
```
//read : Get student by ID
app.get ("/students /: id', asyn (req, res)⇒{
    try {
      const student: await
       student find By Id (req. Params. id);
      if (!student) {
          return res. status (404). json {
              message: "student not found"
          })
      }
    res . status (200) . json (student)
} catch (error) {
    res. status (400) .json {{message
      error . message ()};
}
});
```

```
//update
app. put ("/student /: id : asyn
                  ((req . res) → {
    try {
student find by Id and update (
        req . params . id
```

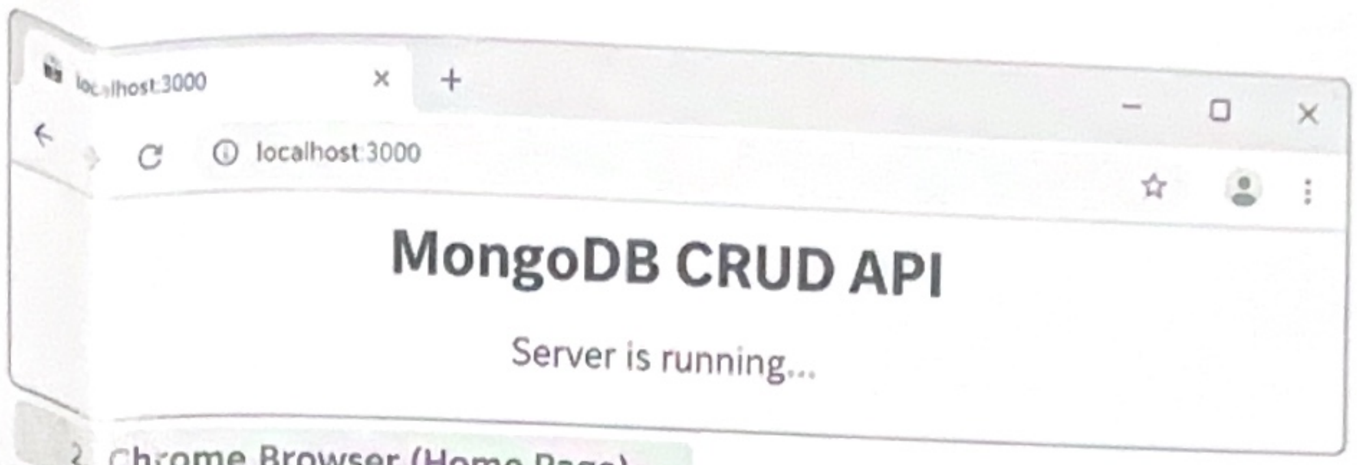
```
if (!student){
    return res.status(404).json({
    message. student not found"
    })
}
catch (error){
    res.status(400).json (message
        error.message});
    }
});

//Delete
app.delete ("/students/:id", async (req,
                                    res) => {
try {
Const student await Student
                findBy Id Delete(
req.params.id
);
if (!student) {
    return res.status(404).
            json({
        message :"student
        not found" });
}
res.status(200).json({
    message : "student deleted
                successfully"
});
} catch (error) {
    res.status(400).json({message
        error.message});
}
});
```

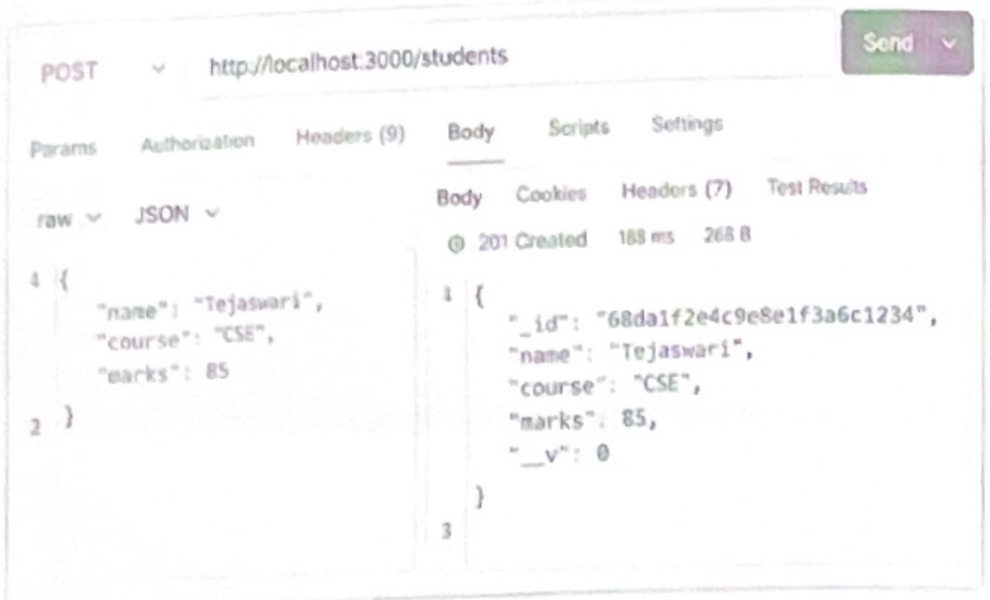
outputs :



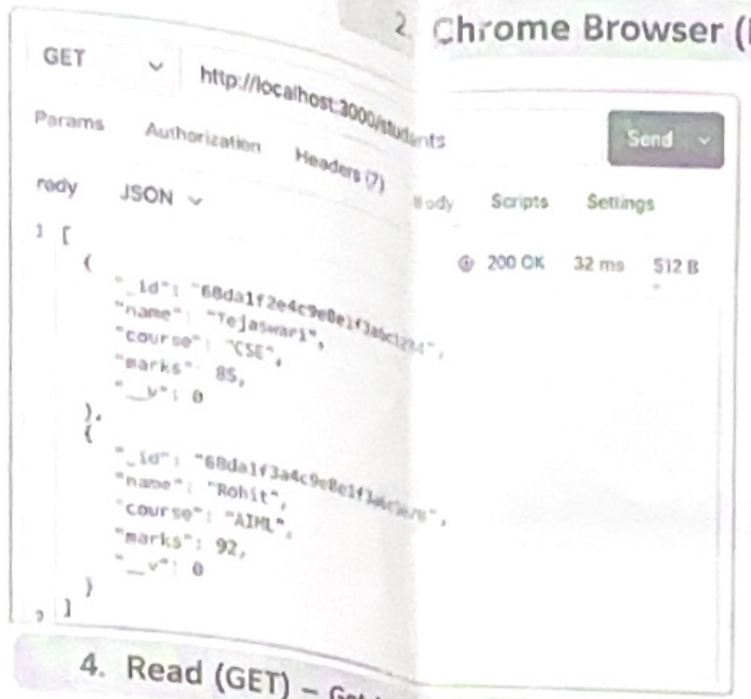
1. Terminal Output



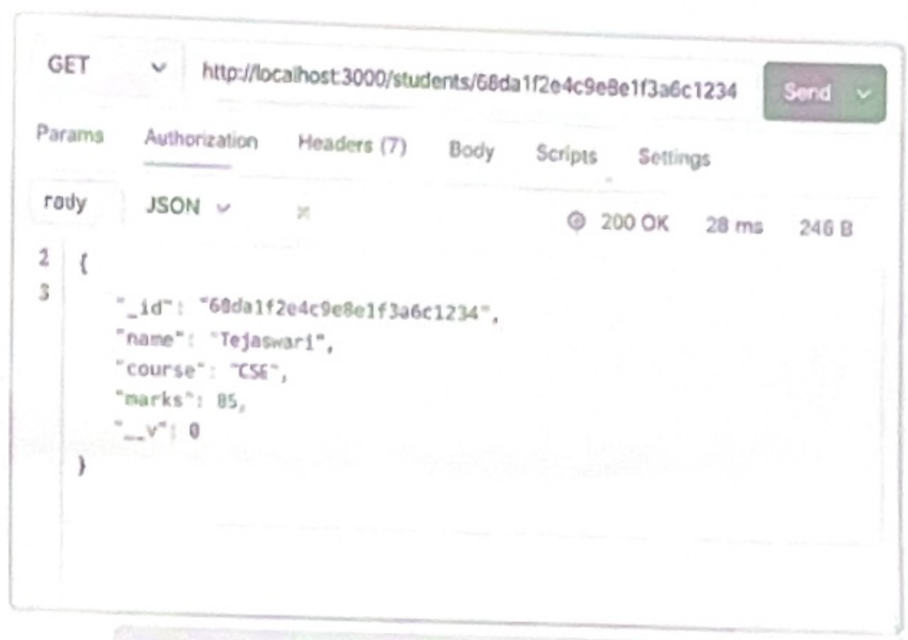
2. Chrome Browser (Home Page)



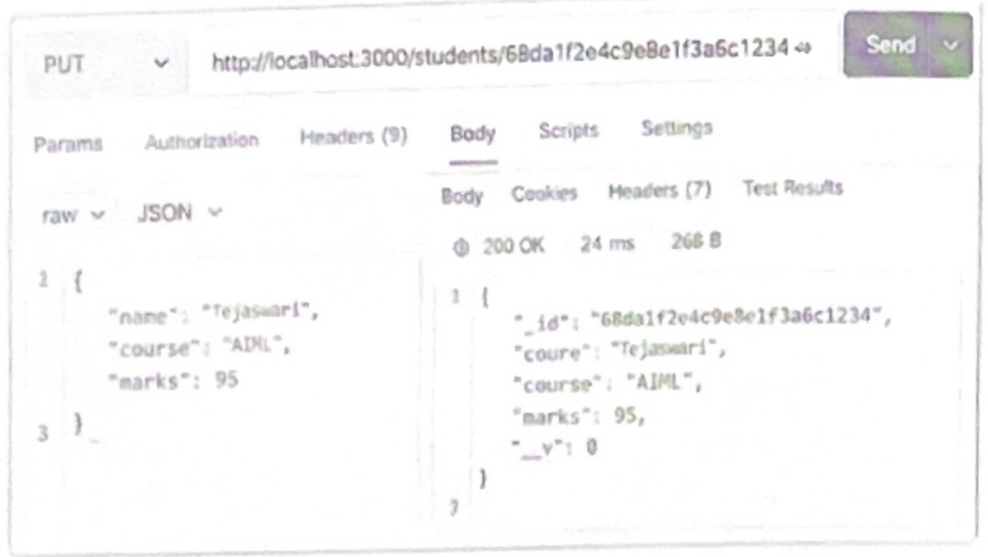
3. Create (POST) – Add Student



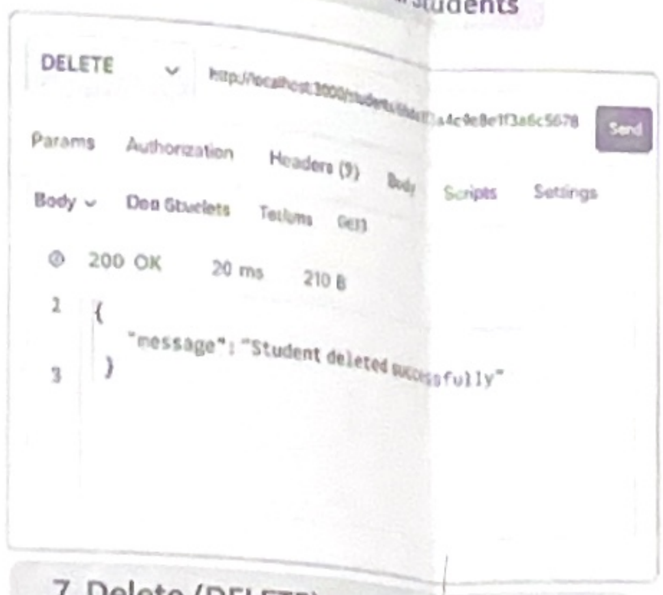
4. Read (GET) – Get All Students



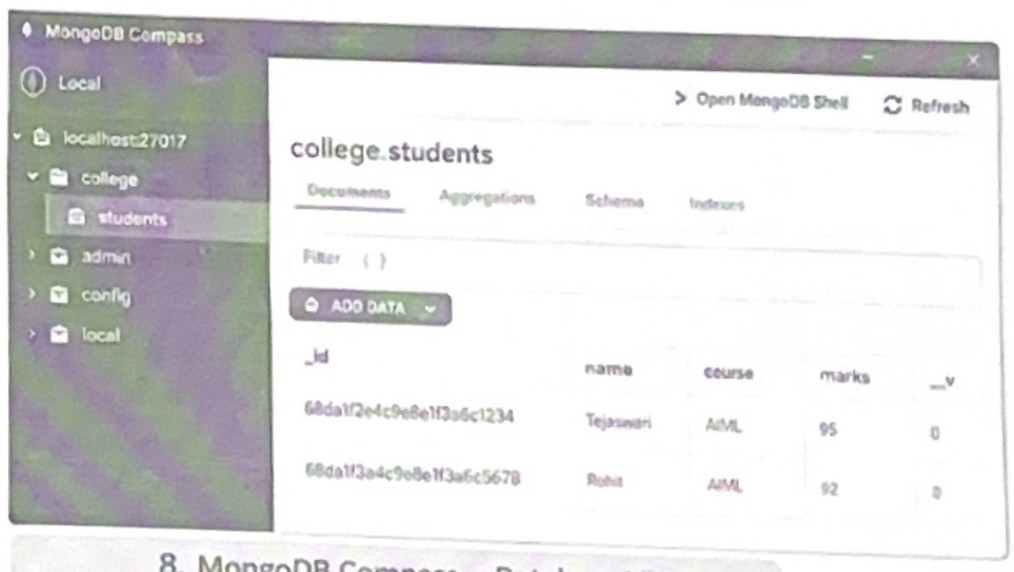
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View